Improving the Binary (Low vs High) Model — Methodology

This document describes the experiment framework added under `research/`, the evaluation protocol it enforces, and the leakage audit that gates every fold. Measured results live in `../RESULTS_BINARY_IMPROVEMENT.md` and, per run, in `outputs/runs/<run_id>/run_summary.md`. For everything in one place — architecture, every experiment, final numbers and limitations — see `PROJECT_RECORD.md`.

The framework is **additive**. It does not modify `serving/`, `serving_3class/`, `backend/model_service.py` or any legacy training script, so the existing 2-class and 3-class demo keeps working exactly as before.

---

1. What changed, and why

The project already had a defensible LOSO pipeline. Three things were missing for the task of *improving* the binary model without fooling ourselves:

1. **No separation between the data used to choose a model and the data used to report its score.** 26-fold LOSO is a good generalization estimate, but if you run thirty experiments and keep the best LOSO number, that number is a selection artifact — the LOSO folds have become a training signal.
2. **No run archive.** Artifacts were written to a shared `outputs/` directory and overwritten, so an experiment could not be traced back to the exact code, config and data that produced it.
3. **No automated leakage assertions.** Leakage-freedom was a property of how the scripts happened to be written, not something checked before each fold.

`research/` addresses all three.

---

2. Evaluation protocol

2.1 The three-way subject split

The 26 subjects are partitioned once, deterministically from the seed:

```
development pool   18 subjects   every experiment, every comparison
lockbox             8 subjects   evaluated exactly once, at the end
```

- **Experiment selection** runs 18-fold LOSO over the development pool. Every architecture, hyperparameter, scheduler, dropout rate and dataset variant is chosen on these folds alone.
- **The lockbox** is never read during the search. After the configuration is frozen, each lockbox subject is evaluated by a model trained only on the 18 development subjects. This is the estimate least contaminated by selection.
- **The headline 26-fold LOSO** is also reported, because it is the protocol the rest of the project uses and keeps the numbers comparable. It is explicitly labelled *selection-informed*: the configuration was chosen using 18 of the 26 subjects it evaluates on, so it should be read alongside the lockbox number, not instead of it.

2.2 Inside one fold

```
held-out subject   ──►  test only. Never in train, val, or any fitted statistic.
remaining subjects ──►  GroupShuffleSplit by subject  ──►  train + validation
```

- Validation subjects are disjoint from training subjects, so the checkpoint is chosen on people the weights never saw.
- `RobustScaler` (EEG) and `StandardScaler` (fNIRS) are refit from scratch on the fold's **training rows only**.
- Checkpoint selection uses **validation macro F1**, with best-weight restore. (`config.py` in the legacy pipeline sets `CHECKPOINT_METRIC = "val_loss"` ; the framework uses macro F1, which is what the project's documentation describes and what a class-balance-sensitive task needs.)

2.3 Model-selection rule, fixed in advance

The search is staged and greedy — each stage varies one axis and the winner is carried forward. The rule is applied mechanically, not by judgement:

1. rank arms by mean development-LOSO macro F1;
2. among arms within one standard error of the leader, take the one with the **fewest parameters**;
3. a challenger replaces the incumbent only if it wins under 1–2, so a tie keeps the simpler incumbent.

Every arm is recorded whether it wins or loses. Losing arms are the negative results and stay in the leaderboard.

---

3. Leakage audit

`research/leakage.py` runs before every fold trains. Any failure raises `LeakageError` and aborts the run — a leaked score is never produced, so it can never be reported.

Per fold

Check	What it proves
`row_index_disjoint`	no row appears in two splits
`subject_disjoint`	train / val / test share no subject
`trial_disjoint`	no trial straddles a split
`window_identity_disjoint`	no `(subject, trial, window_id)` key on both sides
`duplicate_feature_rows`	no byte-identical feature row across splits
`scaler_fit_on_train_only`	the rows used to fit the scalers ⊆ training rows
`heldout_subject_never_fitted`	the held-out subject is absent from train and val
`no_nan_or_inf`	preprocessing produced finite values
`labels_valid`	labels in range, training fold has both classes

Per dataset

Check	What it proves
`per_subject_norm_is_subject_local`	see below
`no_duplicate_rows_in_dataset`	the windows are genuinely distinct samples
`features_finite`	no NaN/Inf survived preprocessing

3.1 The sliding-window question

Windows overlap in time within a trial (10 s windows at 5 s stride overlap by 50%; the dense variant at 1 s stride overlaps by 90%). Overlapping windows share raw signal, so putting two of them on opposite sides of a split leaks.

Grouping by subject makes this structurally impossible: every window of the held-out subject is in the test set and none is anywhere else. The `window_identity_disjoint` and `trial_disjoint` checks assert it explicitly rather than relying on the argument.

3.2 Per-subject normalization — the honest caveat

Each subject's features are z-scored against that subject's own statistics. This was the project's single most important preprocessing decision, and it needs care because it happens before the fold split.

It cannot leak between subjects. `assert_subject_local` proves this numerically rather than in prose: it re-runs the whole preprocessing chain for one subject *from that subject's raw rows in isolation* and checks the result matches the full-dataset computation bit-for-bit. If any cross-subject statistic had crept in, the isolated recomputation would differ. Observed difference: 0.

It is transductive within a subject. Computing a subject's mean and standard deviation needs that subject's whole session. That is standard practice in BCI and it uses no labels — but it means deployment requires a recorded session before the first prediction, not a single window. Three modes are therefore available and compared as an experiment:

Mode	Statistics come from	Deployment story
`transductive`	all of the subject's windows	needs the whole session up front
`calibration`	the first *K* windows only	a short calibration block, then live
`none`	no per-subject normalization	nothing needed

4. What is being varied

Each stage varies one axis from the incumbent.

Stage	Axis	Arms
`S0`	baseline	served architecture + `config.py` hyperparameters
`S1`	batch size / learning rate	64, 128, lr 1e-3, lr 1e-4
`S2`	epoch budget + LR schedule	patience 15/25, cosine, no schedule
`S3`	weight decay	0, 0.003, 0.05, 0.1
`S4`	dropout	0.3/0.3, 0.4/0.5, 0.6/0.7
`S5`	capacity	projection 32/48/192, 1 and 3 residual blocks
`S6`	architecture	CNN, CNN+residual, CNN+residual+attention, wide CNN, MLP+concat
`S7`	modality dropout	0.1, 0.2, asymmetric
`S8`	input noise / label smoothing	0, 0.1, 0.2, smoothing 0.1
`S9`	DANN	lambda 0.1, 0.5, 1.0
`S10`	data + preprocessing	stride, window length, per-subject norm mode, HbR

4.1 Why a 1D CNN is legitimate here

The mentor's CNN suggestion needed a defensible feature layout before it could be tested. The columns are **channel-major**, so each modality block reshapes exactly onto a (channel, feature-type) grid:

```
EEG    540 = 30 electrodes x 18 feature types   (6 bands x {mean, std, slope})
fNIRS  936 = 36 optodes    x 26 feature types   (13 stats x {HbO, HbR})
```

Convolving along the **electrode / optode axis**, with the feature types as the convolution's input channels, shares one filter bank across all channels. That is a genuine inductive bias — look for the same spectral or haemodynamic pattern everywhere on the head — and it cuts the input-stage parameter count by more than an order of magnitude versus a dense `540 → 96` projection (roughly 55K total parameters versus 246K for the MLP baseline).

The assumption it makes is that neighbouring channel indices are spatially related, which is only approximately true for this montage. That is exactly why it is run as a controlled experiment against the MLP on identical folds rather than adopted on the mentor's recommendation alone.

4.2 Modality dropout

During **training only**, each sample's EEG or fNIRS embedding is zeroed with a configurable probability (never both at once). The intent is to stop the head depending on a single modality. `self.training` gates it, so validation, LOSO evaluation and the lockbox always see both modalities.

4.3 DANN

A subject discriminator sits behind a gradient-reversal layer on the fused embedding, with the standard Ganin ramp for lambda. Domain labels are the **training** subjects only, densely re-indexed per fold, so the discriminator never has a class for the held-out subject.

---

5. Run archive

A `RunContext` is created **before any data is read**, so a run that crashes during loading still leaves a directory with its metadata, config and error. Run ids embed a timestamp plus a random suffix and directory creation uses `exist_ok=False`: a run can never overwrite another.

```
outputs/runs/run_<YYYYmmdd>_<HHMMSS>_<hex4>/
├── run_metadata.json      environment, git commit, dataset, seed
├── config.json            the exact resolved configuration
├── dataset_info.json      provenance, schema, class semantics, preprocessing
├── run_status.json        created | running | completed | failed  (+ stage, warnings)
├── run_summary.json/.md   final summary
├── leakage_audit.json     dataset-level audit result
├── metrics/               metrics.json/.csv, per_class, loso, per_subject
├── predictions/           predictions.csv, probabilities.csv
├── confusion_matrix/      json + png
├── curves/                training/validation curves, ROC, PR (json + png)
├── feature_importance/    shap_values.json, feature_ranking.csv, pngs
├── representations/       t-SNE coordinates (csv + json), embeddings, pngs
├── model/                 model.pt, per-fold checkpoints, scaler, schema, labels
├── experiments/           every arm's record, leaderboard.csv, stage_log.json
├── folds/
└── logs/                  training.log, error.log
```

Failed runs are preserved, with `status = failed`, the stage that failed, the exception and a full traceback.

5.1 Predictions are the source of truth

`predictions/predictions.csv` stores one row per evaluated window: subject, trial, window id, fold, true label, predicted label, full probability vector and confidence. Every metric in `metrics.json` — and therefore every number on the Evaluation page — is computed from that file and can be recomputed from it.

5.2 The deployment model

Each run also trains one model on **all** subjects and saves it under `model/`. It has no held-out data, so it carries **no performance claim**; its `model_config.json` says so explicitly. Reported metrics come from the per-fold models under `model/folds/`.

---

6. Metrics

`research/metrics.py` computes, from predictions alone: accuracy, balanced accuracy, macro/weighted/micro precision-recall-F1, Cohen's Kappa, MCC, ROC-AUC (OVR and OVO), macro average precision, log loss, Brier score, top-K accuracy, per-class precision/recall/F1/specificity/support with TP-FP-FN-TN, the confusion matrix, per-subject metrics, and fold mean/std/min/max.

Validity is decided by the data, not by a branch on the model name. Top-K is emitted only when `K < n_classes`; one-vs-one AUC only when `n_classes > 2`. Anything undefined is reported as `null` alongside a stated reason, which the UI renders instead of a number — for a binary model, Top-3 shows *"Not applicable: K=3 is not smaller than the number of classes (2)."*

---

7. Frontend

Two new pages, both driven entirely by run artifacts.

`/evaluation` — Evaluation Metrics. A run selector controls the whole page; selecting a run reloads every section from that run alone, so run A's metrics can never appear beside run B's plots. Sections: model/protocol/dataset, held-out performance (with lockbox and generalization gap), classification metrics, per-class metrics, confusion matrix, ROC, precision-recall, per-subject performance, training vs validation curves, feature importance, t-SNE, experiment comparison, run details. Class names, class count, chance level and metric validity all come from the selected run's own artifacts.

`/runs` — Run History. Every run in the archive with status, dataset, protocol, metrics and warnings; filterable; each row opens that run's evaluation. Failed runs are listed with their failure stage and message. A run that recorded itself as running but has not updated in 30 minutes is flagged *interrupted* in this view only — the status on disk is left untouched.

Backend endpoints

```
GET /api/runs                              run history (filter: mode, status)
GET /api/runs/{id}                         metadata, config, dataset, audit, manifest
GET /api/runs/{id}/metrics                 metrics + explanations + LOSO + per-class
GET /api/runs/{id}/confusion-matrix
GET /api/runs/{id}/curves                  ROC + PR points
GET /api/runs/{id}/training-history        per-fold and fold-averaged curves
GET /api/runs/{id}/per-subject
GET /api/runs/{id}/feature-importance      SHAP
GET /api/runs/{id}/representations         t-SNE + subject-invariance probe
GET /api/runs/{id}/experiments             leaderboard + stage log
GET /api/runs/{id}/experiments/{exp_id}
GET /api/runs/{id}/predictions             paged raw predictions
GET /api/runs/{id}/summary                 run_summary.json
GET /api/runs/{id}/report                  run_summary.md
GET /api/runs/{id}/figure?path=...         a PNG the run rendered (path-guarded)
```

Every one returns bytes a run wrote to disk. Nothing is computed in the request path, so the API cannot disagree with the artifacts.

---

8. Reproducing

```
# full campaign (several hours on CPU)
.venv/bin/python run_campaign.py --tag my-run

# a single stage
.venv/bin/python run_campaign.py --stages S6

# smoke test: reduced cohort and epoch budget, every code path exercised
.venv/bin/python run_campaign.py --quick --limit-subjects 7 --n-lockbox 3 --max-epochs 6
```

`--limit-subjects` and `--max-epochs` record a warning into the run, and that warning is shown on the Evaluation page, because a reduced-cohort run is not a valid performance estimate.

The datasets are referenced from `data/` (symlinks into `ZDATASET/`):

Key	File	Windows	Geometry
`sliding_10s_5s`	`tuberlin_sliding.csv`	7,150 (4,576 binary)	10 s window, 5 s stride
`sliding_10s_1s`	`tuberlin_sliding_augmented.csv`	33,150 (21,216 binary)	10 s window, 1 s stride
`sliding_8s_4s`	`tuberlin_sliding_8s_4s.csv`	9,100 (5,824 binary)	8 s window, 4 s stride

Binary framing keeps `Cognitive_Load ∈ {0, 2}` — 0-back as **Low**, 3-back as **High** — and drops the 2-back middle class. Both classes are exactly balanced.